

```
=====
FILE: karscan.sh
=====
#!/usr/bin/env bash
# =====
# karscan.sh – Bash-Based Security Assessment Tool
# Instant Software Solutions – Security Track (SEC-BASH-092226)
#
# A mini automated security assessment framework: reconnaissance, port &
# service enumeration, service-specific security checks, and a structured,
# evidence-based report – with the operator choosing the approach for every
# stage instead of the tool guessing silently.
#
# Usage:
# ./karscan.sh <target> Interactive single-target scan
# ./karscan.sh -f targets.txt Batch scan (auto mode, one report per host)
# ./karscan.sh --auto <target> Non-interactive single target (uses defaults)
# ./karscan.sh --help | --version
#
# Only scan hosts you own or are explicitly authorized to test.
# =====

set -uo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

VERSION="1.0.0"

# ---- exit codes -----
EXIT_OK=0
EXIT_USAGE=1
EXIT_MISSING_DEP=2
EXIT_UNREACHABLE=3
EXIT_INTERRUPTED=130

# ---- global state -----
declare -A AVAILABLE_TOOLS
declare -A OPEN_PORTS
declare -A PORT_VERSION
FINDINGS=()
AUTO_MODE=0
TARGET=""
TARGET_IP=""
TARGETS_FILE=""
FW_EVADE="0"
SCAN_PROTOCOL="tcp"
STEALTH_PROFILE="pentest"
SCAN_TECHNIQUE_LABEL=""
STEALTH_LABEL=""

# ---- load libraries & modules -----
# shellcheck source=/dev/null
source "$SCRIPT_DIR/lib/colors.sh"
source "$SCRIPT_DIR/lib/logging.sh"
source "$SCRIPT_DIR/lib/utils.sh"
source "$SCRIPT_DIR/modules/recon.sh"
source "$SCRIPT_DIR/modules/evasion.sh"
source "$SCRIPT_DIR/modules/portscan.sh"
source "$SCRIPT_DIR/modules/ftp.sh"
source "$SCRIPT_DIR/modules/ssh.sh"
source "$SCRIPT_DIR/modules/smb.sh"
source "$SCRIPT_DIR/modules/smtp.sh"
source "$SCRIPT_DIR/modules/dns.sh"
source "$SCRIPT_DIR/modules/http.sh"
source "$SCRIPT_DIR/modules/report.sh"
```

```

trap 'printf "\n%s Interrupted – exiting.\n" "$SYM_FAIL"; exit $EXIT_INTERRUPTED' INT TERM

# -----
print_usage() {
    cat <<EOF
Usage: ./karscan.sh <target>
      ./karscan.sh -f <targets_file>
      ./karscan.sh [--auto] <target>
      ./karscan.sh --help
      ./karscan.sh --version

    <target>          IP address or hostname of an authorized lab host
    -f, --file <file> Text file with one target per line (batch mode, auto settings)
    --auto           Skip interactive menus for a single target and use safe defaults
    -h, --help       Show this help and exit
    -v, --version     Show version and exit

Examples:
  ./karscan.sh 192.168.56.105
  ./karscan.sh -f targets.txt
  ./karscan.sh --auto 192.168.56.105

Only scan targets you own or are explicitly authorized to test.
EOF
}

print_version() {
    printf 'karscan.sh version %s\n' "$VERSION"
}

# -----
# Maps a detected service name (as nmap reports it) to a check_* function.
service_to_module() {
    case "$1" in
        ftp) echo "check_ftp" ;;
        ssh) echo "check_ssh" ;;
        microsoft-ds|netbios-ssn|smb) echo "check_smb" ;;
        smtp) echo "check_smtp" ;;
        domain) echo "check_dns" ;;
        http|http-proxy|http-alt) echo "check_http" ;;
        https|ssl/http) echo "check_https" ;;
        *) echo "" ;;
    esac
}

service_display_name() {
    case "$1" in
        ftp) echo "FTP" ;;
        ssh) echo "SSH" ;;
        microsoft-ds|netbios-ssn|smb) echo "SMB" ;;
        smtp) echo "SMTP" ;;
        domain) echo "DNS" ;;
        http|http-proxy|http-alt) echo "HTTP" ;;
        https|ssl/http) echo "HTTPS" ;;
        *) echo "$1" ;;
    esac
}

dispatch_enumeration() {
    section "Stage 3/4 – Automated Enumeration"

    # Build the list of ports whose service we actually know how to check.
    local checkable_ports=() p svc fn
    for p in "${!OPEN_PORTS[@]}; do

```

```

        svc="${OPEN_PORTS[$p]}"
        fn="$(service_to_module "$svc")"
        [[ -n "$fn" ]] && checkable_ports+=("$p")
done

if (( ${#checkable_ports[@]} == 0 )); then
    log_warn "No open ports matched a known service module – nothing to enumerate"
    return
fi

local -a sorted_ports
mapfile -t sorted_ports < <(printf '%s\n' "${checkable_ports[@]}" | sort -n)

printf '    Services matched to a check module:\n'
for p in "${sorted_ports[@]"; do
    printf '        - port %-6s %s\n' "$p" "$(service_display_name "${OPEN_PORTS[$p]}")"
done

ask_menu "Run automated service checks?" 1 \
    "Run all matched checks automatically (recommended)" \
    "Let me choose which checks to run" \
    "Skip automated enumeration"
local enum_choice="$MENU_CHOICE"

[[ "$enum_choice" == "3" ]] && { log_warn "Automated enumeration skipped by user choice";
return; }

local -a to_run=()
if [[ "$enum_choice" == "1" ]]; then
    to_run=("${sorted_ports[@]}")
else
    for p in "${sorted_ports[@]"; do
        if ask_yes_no "Run $(service_display_name "${OPEN_PORTS[$p]}") check on port $p?" 1;
then
            to_run+=("$p")
        fi
    done
fi

for p in "${to_run[@]"; do
    svc="${OPEN_PORTS[$p]}"
    fn="$(service_to_module "$svc")"
    log_ok "$(service_display_name "$svc") detected on port $p"
    log_info "Starting $(service_display_name "$svc") enumeration..."
    "$fn" "$TARGET" "$p"
done
}

# -----
run_single_target() {
    local target="$1"
    TARGET="$target"
    TARGET_IP=""
    OPEN_PORTS=()
    PORT_VERSION=()
    FINDINGS=()

    local ts safe_name
    ts="$(date '+%Y%m%d_%H%M%S')"
    safe_name="$(printf '%s' "$target" | tr -c 'A-Za-z0-9._-' '_')"
    REPORT_DIR="$SCRIPT_DIR/reports/${safe_name}_${ts}"
    mkdir -p "$REPORT_DIR"

    SCAN_FILE="$REPORT_DIR/scan.txt"
    FINDINGS_FILE="$REPORT_DIR/findings.txt"

```

```

SUMMARY_FILE="$REPORT_DIR/summary.txt"
HTML_FILE="$REPORT_DIR/report.html"
LOG_FILE="$REPORT_DIR/karscan.log"
: > "$SCAN_FILE"; : > "$LOG_FILE"

banner "KarScan - target: $target"
log_info "Report directory: $REPORT_DIR"

local start_ts end_ts
start_ts="$(date +%s)"

if [[ "$AUTO_MODE" == "1" ]]; then
    RECON_LEVEL=2; SCAN_TYPE=1; REPORT_MODE=2
    evasion_defaults
else
    recon_menu
    portscan_menu "$target"
    evasion_menu "$target"
    report_menu
fi

run_recon "$target"
local recon_rc=?
if [[ $recon_rc -eq $EXIT_UNREACHABLE ]]; then
    end_ts="$(date +%s)"; SCAN_DURATION=$(( end_ts - start_ts ))
    generate_reports
    return $EXIT_UNREACHABLE
fi

run_portscan "$target"
dispatch_enumeration

end_ts="$(date +%s)"
SCAN_DURATION=$(( end_ts - start_ts ))
generate_reports
return $EXIT_OK
}

# -----
main() {
    if (( $# == 0 )); then
        print_usage
        exit $EXIT_USAGE
    fi

    while (( $# > 0 )); do
        case "$1" in
            -h|--help) print_usage; exit $EXIT_OK ;;
            -v|--version) print_version; exit $EXIT_OK ;;
            --auto) AUTO_MODE=1; shift ;;
            -f|--file) TARGETS_FILE="${2:-}"; shift 2 ;;
            -*)
                log_fail "Unknown option: $1"
                print_usage
                exit $EXIT_USAGE
                ;;
            *)
                TARGET="$1"; shift ;;
        esac
    done

    check_dependencies

    if [[ -n "$TARGETS_FILE" ]]; then
        [[ -f "$TARGETS_FILE" ]] || { log_fail "Targets file not found: $TARGETS_FILE"; exit

```

```

$EXIT_USAGE; }
    AUTO_MODE=1
    local overall_rc=$EXIT_OK
    local line
    while IFS= read -r line || [[ -n "$line" ]]; do
        line="$(echo "$line" | xargs)"
        [[ -z "$line" || "$line" == \#* ]] && continue
        run_single_target "$line" || overall_rc=$?
        printf '\n'
    done < "$TARGETS_FILE"
    exit $overall_rc
fi

if [[ -z "$TARGET" ]]; then
    print_usage
    exit $EXIT_USAGE
fi

run_single_target "$TARGET"
exit $?
}

main "$@"

=====
FILE: lib/colors.sh
=====
#!/usr/bin/env bash
# =====
# colors.sh – terminal colors, symbols, and small UI helpers for KarScan
# =====

# Only emit color codes when writing to an actual terminal.
if [[ -t 1 ]]; then
    C_RESET=$'\033[0m'
    C_BOLD=$'\033[1m'
    C_DIM=$'\033[2m'
    C_RED=$'\033[31m'
    C_GREEN=$'\033[32m'
    C_YELLOW=$'\033[33m'
    C_BLUE=$'\033[34m'
    C_MAGENTA=$'\033[35m'
    C_CYAN=$'\033[36m'
    C_WHITE=$'\033[97m'
    C_GRAY=$'\033[90m'
else
    C_RESET=""; C_BOLD=""; C_DIM=""; C_RED=""; C_GREEN="";
    C_YELLOW=""; C_BLUE=""; C_MAGENTA=""; C_CYAN=""; C_WHITE=""; C_GRAY="";
fi

SYM_OK="${C_GREEN}[+]${C_RESET}"
SYM_INFO="${C_CYAN}[*]${C_RESET}"
SYM_WARN="${C_YELLOW}[!]${C_RESET}"
SYM_FAIL="${C_RED}[-]${C_RESET}"
SYM_ASK="${C_MAGENTA}[?]${C_RESET}"

hr() {
    printf '%s\n' "${C_GRAY}
    _____${C_RESET}"
}

banner() {
    local title="$1"
    hr
    printf '  %s%s\n' "${C_BOLD}${C_WHITE}" "$title" "${C_RESET}"
}

```

```

    hr
}

section() {
    local title="$1"
    printf '\n%s%s▶ %s%s\n' "${C_BOLD}" "${C_CYAN}" "$title" "${C_RESET}"
}

=====
FILE: lib/logging.sh
=====
#!/usr/bin/env bash
# =====
# logging.sh – unified logging: console output + persistent log file
# =====
# Requires colors.sh to be sourced first (uses SYM_* / C_* variables).
# Requires LOG_FILE to be set by the caller before any log_* call.

_log_ts() { date '+%Y-%m-%d %H:%M:%S'; }

_log_write() {
    # Always append a plain-text line to the log file, regardless of console color.
    [[ -n "${LOG_FILE:-}" ]] && printf "[%s] %s\n" "$(_log_ts)" "$1" >> "$LOG_FILE"
}

log_ok() {
    printf '%s %s\n' "$SYM_OK" "$1"
    _log_write "OK    | $1"
}

log_info() {
    printf '%s %s\n' "$SYM_INFO" "$1"
    _log_write "INFO  | $1"
}

log_warn() {
    printf '%s %s\n' "$SYM_WARN" "$1" >&2
    _log_write "WARN  | $1"
}

log_fail() {
    printf '%s %s\n' "$SYM_FAIL" "$1" >&2
    _log_write "FAIL  | $1"
}

log_finding() {
    # Structured, evidence-backed finding. Appears in console, log, and findings buffer.
    # Usage: log_finding "<severity>" "<title>" "<evidence>" "<recommendation>"
    local severity="$1" title="$2" evidence="$3" recommendation="$4"
    local sev_color="$C_YELLOW"
    case "$severity" in
        HIGH)    sev_color="$C_RED" ;;
        MEDIUM) sev_color="$C_YELLOW" ;;
        LOW)     sev_color="$C_BLUE" ;;
        INFO)    sev_color="$C_GRAY" ;;
    esac
    printf '%s %s[%s]%s %s\n' "$SYM_WARN" "$sev_color" "$severity" "${C_RESET}" "$title"
    printf '    %sEvidence:%s %s\n' "${C_DIM}" "${C_RESET}" "$evidence"

    FINDINGS+=("${severity}|${title}|${evidence}|${recommendation}")
    _log_write "FINDING [$severity] | $title | Evidence: $evidence | Recommendation:
$recommendation"
}

=====

```

```

FILE: lib/utils.sh
=====
#!/usr/bin/env bash
# =====
# utils.sh - dependency checks, validation helpers, and the interactive
#           "ask before every step" menu system.
# =====

# ---- dependency handling -----

REQUIRED_TOOLS=(nmap ping curl dig)
OPTIONAL_TOOLS=(ftp smbclient enum4linux nc whois traceroute)

check_dependencies() {
    local missing=()
    local tool
    for tool in "${REQUIRED_TOOLS[@]"; do
        if ! command -v "$tool" &>/dev/null; then
            missing+=("$tool")
        fi
    done

    if (( ${#missing[@]} > 0 )); then
        for tool in "${missing[@]"; do
            log_fail "$tool is not installed"
        done
        log_info "Install them, e.g.: sudo apt install ${missing[*]}"
        exit "$EXIT_MISSING_DEP"
    fi

    for tool in "${OPTIONAL_TOOLS[@]"; do
        if ! command -v "$tool" &>/dev/null; then
            AVAILABLE_TOOLS["$tool"]=0
        else
            AVAILABLE_TOOLS["$tool"]=1
        fi
    done
}

tool_available() {
    # Usage: tool_available smbclient && ...
    local tool="$1"
    [[ "${AVAILABLE_TOOLS[$tool]:-0}" == "1" ]]
}

# ---- validation -----

is_ip() {
    local ip="$1"
    [[ "$ip" =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]] || return 1
    local octet
    for octet in ${ip//./ }; do
        (( octet >= 0 && octet <= 255 )) || return 1
    done
    return 0
}

resolve_target() {
    # Prints the IP for a hostname, or echoes the IP back unchanged.
    local target="$1"
    if is_ip "$target"; then
        printf '%s' "$target"
    else
        getent hosts "$target" 2>/dev/null | awk '{print $1; exit}'
    fi
}

```

```

}

check_reachable() {
    local target="$1"
    ping -c 2 -W 2 "$target" &>/dev/null
}

# ---- interactive menu system -----
# Every "ask before a step" prompt in KarScan goes through ask_menu so the
# behaviour (numbering, validation, non-interactive fallback) stays consistent.
#
# Usage: ask_menu "<prompt title>" default_index "opt1" "opt2" "opt3" ...
# Returns the chosen option's 1-based index via $MENU_CHOICE.
# When AUTO_MODE=1 (non-interactive / batch mode), it silently picks the
# supplied default and prints what it picked, so logs stay legible.

ask_menu() {
    local title="$1"; shift
    local default_idx="$1"; shift
    local options=("$@")
    local i choice

    printf '\n%s %s%s\n' "$SYM_ASK" "$C_BOLD" "$title" "$C_RESET"
    for i in "${!options[@]}; do
        local marker=" "
        (( i + 1 == default_idx )) && marker="${C_GREEN}*${C_RESET}"
        printf '    %s [%d] %s\n' "$marker" "$((i + 1))" "${options[$i]}"
    done

    if [[ "$AUTO_MODE" == "1" ]]; then
        printf '    %s→ auto mode: using default [%d] %s%s\n' "$C_DIM" "$default_idx"
        "${options[$((default_idx - 1))]}" "$C_RESET"
        MENU_CHOICE="$default_idx"
        return 0
    fi

    while true; do
        read -r -p "    Choice [default ${default_idx}]: " choice
        [[ -z "$choice" ]] && choice="$default_idx"
        if [[ "$choice" =~ ^[0-9]+$ ]] && (( choice >= 1 && choice <= ${#options[@]} )); then
            MENU_CHOICE="$choice"
            return 0
        fi
        printf '    %s invalid choice, try again\n' "$SYM_WARN"
    done
}

ask_yes_no() {
    # Usage: ask_yes_no "Question?" default_yes(0/1) -> returns 0 for yes, 1 for no
    local prompt="$1" default="$2" reply
    local hint="y/N"
    (( default == 1 )) && hint="Y/n"

    if [[ "$AUTO_MODE" == "1" ]]; then
        return $(( default == 1 ? 0 : 1 ))
    fi

    read -r -p "$(printf '%s %s [%s]: ' "$SYM_ASK" "$prompt" "$hint")" reply
    reply="${reply,,}"
    if [[ -z "$reply" ]]; then
        return $(( default == 1 ? 0 : 1 ))
    fi
    [[ "$reply" == "y" || "$reply" == "yes" ]]
}

```

```

human_duration() {
    local secs="$1"
    printf '%dm%02ds' "$(( secs / 60 ))" "$(( secs % 60 ))"
}

=====
FILE: modules/dns.sh
=====
#!/usr/bin/env bash
# =====
# dns.sh – DNS service checks
# =====

check_dns() {
    local target="$1" port="$2"
    log_info "Starting DNS checks on port $port..."

    if [[ -z "${DNS_DOMAIN:-}" ]]; then
        DNS_DOMAIN=""
        if ask_yes_no "Do you already know the domain name and want to enter it yourself?" 0;
then
            read -r -p "    $SYM_ASK Domain name to test for zone transfer (blank = skip): "
DNS_DOMAIN
        elif ask_yes_no "Do you want KarScan to try to auto-discover the domain name instead?" 1;
then
            log_info "Attempting to auto-discover a domain name (reverse DNS / SOA)..."

            local ptr
            ptr="$(timeout 6 dig @"$target" -p "$port" -x "$target" +short 2>/dev/null | head -
n1)"

            if [[ -n "$ptr" ]]; then
                DNS_DOMAIN="$(echo "$ptr" | sed -E 's/^[^.]+\./; s/\.$///')"
            fi

            if [[ -z "$DNS_DOMAIN" ]]; then
                local guess soa
                for guess in localdomain local vulnos.local lan; do
                    soa="$(timeout 4 dig @"$target" -p "$port" "$guess" SOA +short 2>/dev/null)"
                    if [[ -n "$soa" ]]; then
                        DNS_DOMAIN="$guess"
                        break
                    fi
                done
            fi

            if [[ -z "$DNS_DOMAIN" ]] && -f /etc/resolv.conf ]; then
                DNS_DOMAIN="$(awk '/^search|^domain/{print $2; exit}' /etc/resolv.conf 2>/dev/
null)"
            fi

            if [[ -n "$DNS_DOMAIN" ]]; then
                log_ok "Domain name auto-discovery succeeded: $DNS_DOMAIN"
            else
                log_warn "Domain name auto-discovery failed"
            fi
        else
            log_warn "Domain name step skipped by user choice"
        fi
    fi

    if [[ -z "$DNS_DOMAIN" ]]; then
        log_warn "No domain provided – skipping zone transfer check"
        return
    fi
    log_finding "INFO" "Domain name identified for DNS testing on port $port" \

```

```

"$DNS_DOMAIN" \
    "No action required – this is the domain name KarScan used to test the DNS server for a
zone transfer (AXFR)."
```

```

local axfr_out
axfr_out="$(timeout 10 dig @$target" -p "$port" "$DNS_DOMAIN" AXFR +time=5 2>&1)"
printf '\n=== DNS AXFR check: dig @%s -p %s %s AXFR ===\n%s\n' "$target" "$port"
"$DNS_DOMAIN" "$axfr_out" >> "$SCAN_FILE"

if printf '%s' "$axfr_out" | grep -qiE 'Transfer failed|connection refused|communications
error|timed out'; then
    log_ok "Zone transfer refused/failed for $DNS_DOMAIN"
    return
fi

local record_count
record_count="$(printf '%s' "$axfr_out" | grep -cE '^[^;].*\s(IN|CH)\s')"
```

```

if (( record_count > 0 )); then
    local sample
    sample="$(printf '%s' "$axfr_out" | grep -E '^[^;].*\s(IN|CH)\s' | head -n 5 | tr '\n'
',' | sed 's/;/ /g')"
```

```

    log_finding "HIGH" "DNS zone transfer (AXFR) allowed for $DNS_DOMAIN on port $port" \
        "dig @$target -p $port $DNS_DOMAIN AXFR -> returned $record_count records. Sample:
${sample:0:200}" \
        "Restrict AXFR to authorized secondary name servers only (allow-transfer in BIND, or
equivalent)."
```

```

else
    log_ok "Zone transfer returned no usable records"
fi
}

=====
FILE: modules/evasion.sh
=====
#!/usr/bin/env bash
# =====
# evasion.sh – Pre-scan profiling stage (ADDITIVE, runs before Stage 2)
# =====
# Asks three extra questions before the port scan and stores the answers in
# globals that portscan.sh reads to adjust the nmap command it builds:
#
#   FW_EVADE           "1" / "0"           – attempt firewall evasion (Xmas scan)?
#   SCAN_PROTOCOL      "tcp" / "udp"        – which protocol family to scan
#   STEALTH_PROFILE    "redteam" / "pentest" – how much extra stealth to layer on
#
# Defaults ("0" / "tcp" / "pentest") reproduce the exact old scan behaviour,
# so this stage only ever *adds* capability – it never changes what already
# worked when the operator just accepts the defaults (or in --auto mode).

evasion_defaults() {
    FW_EVADE="0"
    SCAN_PROTOCOL="tcp"
    STEALTH_PROFILE="pentest"
    SCAN_TECHNIQUE_LABEL=""
    STEALTH_LABEL=""
}

evasion_menu() {
    local target="$1"

    if [[ "$AUTO_MODE" == "1" ]]; then
        evasion_defaults
        log_info "Auto mode: firewall evasion off, TCP, Penetration Test profile (unchanged

```

```

default behavior)"
    return 0
fi

section "Pre-Scan Profile - $target"

ask_menu "Do you want to try to evade the firewall/IDS?" 2 \
    "Yes - use a TCP Xmas scan (FIN+PSH+URG flags) to try to slip past simple packet filters"
\
    "No - scan normally (recommended unless you know a firewall is in the way)"
[[ "$MENU_CHOICE" == "1" ]] && FW_EVADE="1" || FW_EVADE="0"

ask_menu "Which protocol do you want to scan?" 1 \
    "TCP - the standard choice for this assignment's target services" \
    "UDP - scan UDP ports instead (slower; needed for services like DNS/SNMP/TFTP over UDP)"
if [[ "$MENU_CHOICE" == "2" ]]; then
    SCAN_PROTOCOL="udp"
else
    SCAN_PROTOCOL="tcp"
fi

ask_menu "What's your operational profile for this scan?" 2 \
    "Red Team - prioritize staying hidden: slow timing, packet fragmentation, decoys,
randomized order" \
    "Penetration Test - thoroughness over stealth: run the scan exactly as configured, at
normal speed"
if [[ "$MENU_CHOICE" == "1" ]]; then
    STEALTH_PROFILE="redteam"
else
    STEALTH_PROFILE="pentest"
fi

printf '\n  %sProfile selected:%s evasion=%s, protocol=%s, mode=%s\n' \
    "$C_DIM" "$C_RESET" \
    "${[[ "$FW_EVADE" == "1" ]] && echo yes || echo no}" \
    "${SCAN_PROTOCOL^^}" \
    "${[[ "$STEALTH_PROFILE" == "redteam" ]] && echo "Red Team" || echo "Penetration Test")"
}

# Mutates the nmap argument array (passed by name / nameref) built by
# portscan.sh's existing _build_nmap_args, layering the evasion/protocol/
# stealth choices on top without touching that function's own case logic.
apply_evasion_overrides() {
    local -n _args="$1"
    local kept=() flag

    if [[ "$SCAN_PROTOCOL" == "udp" ]]; then
        # UDP scanning is a different technique entirely - swap out whichever
        # TCP technique flag _build_nmap_args chose (-sS/-sT) for -sU.
        for flag in "${_args[@]}"; do
            case "$flag" in
                -sS|-sT|-sX) continue ;;
                *) kept+=("$flag") ;;
            esac
        done
        _args=(-sU "${kept[@]}")
    fi

    if [[ "$FW_EVADE" == "1" ]]; then
        # Xmas doesn't exist for UDP (UDP has no flags to set) - apply the
        # closest generic evasion instead: fragmentation + padded length.
        _args+=(-f --data-length 16)
        SCAN_TECHNIQUE_LABEL="UDP scan (-sU) with generic evasion (-f fragmentation, --data-
length 16) - Xmas doesn't apply to UDP, so packet fragmentation/padding was used instead"
    else
        SCAN_TECHNIQUE_LABEL="UDP scan (-sU)"
    fi
}

```

```

fi

elif [[ "$FW_EVADE" == "1" ]]; then
    for flag in "${_args[@]"; do
        case "$flag" in
            -sS|-sT) continue ;;
            *) kept+=("$flag") ;;
        esac
    done
    if (( EUID == 0 )); then
        _args=(-sX "${kept[@]}")
        SCAN_TECHNIQUE_LABEL="TCP Xmas scan (-sX: FIN+PSH+URG flags set, no SYN/ACK) –
firewall evasion mode"
    else
        _args=(-sT "${kept[@]}")
        SCAN_TECHNIQUE_LABEL="TCP connect scan (-sT) – Xmas scan needs root privileges, fell
back automatically"
        log_warn "Xmas scan requires root – falling back to TCP connect scan"
    fi
else
    SCAN_TECHNIQUE_LABEL="TCP scan (technique as selected in the port-scan menu)"
fi

if [[ "$STEALTH_PROFILE" == "redteam" ]]; then
    # Avoid stacking a duplicate -f / --data-length if the UDP-evasion
    # branch above already added one – keep the args clean either way.
    if [[ "$SCAN_PROTOCOL" == "udp" && "$FW_EVADE" == "1" ]]; then
        _args+=(-T1 --randomize-hosts)
        STEALTH_LABEL="Red Team – maximum practical stealth: -T1 (paranoid timing, minimizes
IDS pattern triggers), --randomize-hosts (randomizes scan order); packet fragmentation (-f) and
padding (--data-length 16) were already applied by the UDP evasion step above"
    else
        _args+=(-T1 -f --randomize-hosts --data-length 24)
        STEALTH_LABEL="Red Team – maximum practical stealth: -T1 (paranoid timing, minimizes
IDS pattern triggers), -f (fragments packets across multiple IP datagrams), --randomize-hosts
(randomizes scan order), --data-length 24 (pads packets to obscure the nmap fingerprint)"
    fi
else
    _args+=(-T3)
    STEALTH_LABEL="Penetration Test – standard nmap timing (-T3), no additional stealth
overhead, prioritizing speed and completeness"
fi
}

# One-line + multi-line description used by the scan-profile banner in
# scan.txt / summary.txt / report.html.
scan_profile_banner_text() {
    cat <<PROFILE
Firewall evasion requested: $([[ "$FW_EVADE" == "1" ]] && echo "Yes (Xmas scan)" || echo "No")
Protocol:                ${SCAN_PROTOCOL^^}
Operational profile:     $([[ "$STEALTH_PROFILE" == "redteam" ]] && echo "Red Team (maximum
stealth)" || echo "Penetration Test (standard)")
Scan technique used:     ${SCAN_TECHNIQUE_LABEL}
Stealth measures applied: ${STEALTH_LABEL}
PROFILE
}

=====
FILE: modules/ftp.sh
=====
#!/usr/bin/env bash
# =====
# ftp.sh – FTP service checks
# =====

```

```

check_ftp() {
    local target="$1" port="$2"
    log_info "Starting FTP enumeration on port $port..."

    if ! tool_available ftp; then
        log_warn "ftp client not installed – skipping anonymous login check (install: sudo apt
install ftp)"
        return
    fi

    local response
    response="$(
        printf 'user anonymous anonymous@karscan.local\npassword\nls\nbye\n' | \
        timeout 10 ftp -inv "$target" "$port" 2>&1
    )"
    printf '\n=== FTP check on %s:%s ===\n%s\n' "$target" "$port" "$response" >> "$SCAN_FILE"

    if printf '%s' "$response" | grep -qiE '230 |login successful|anonymous access granted'; then
        local listing
        listing="$(printf '%s' "$response" | grep -A20 -iE '^(150|226|229)|drwx|^-rw' | head -n
10 | tr '\n' ';' | sed 's/;; /g')"
        [[ -z "$listing" ]] && listing="(login succeeded but directory listing was empty)"
        log_finding "HIGH" "Anonymous FTP login allowed on port $port" \
            "ftp $target $port -> login as \"anonymous\" succeeded. Listing: ${listing:0:200}" \
            "Disable anonymous FTP access unless explicitly required; if required, restrict to
read-only and a dedicated non-sensitive directory."
        else
            log_ok "Anonymous FTP login rejected"
        fi
    }
}

```

```

=====
FILE: modules/http.sh
=====

```

```

#!/usr/bin/env bash

```

```

# =====
# http.sh – HTTP/HTTPS basic enumeration
# =====

```

```

check_http() {
    local target="$1" port="$2"
    local scheme="http"
    [[ "$port" == "443" || "$port" == "8443" ]] && scheme="https"
    local base_url="${scheme}://${target}:${port}"

    log_info "Starting HTTP enumeration on $base_url..."

    # --- headers ---
    local headers
    headers="$(timeout 10 curl -skI --max-time 8 "$base_url/" 2>&1)"
    printf '\n=== HTTP headers: %s ===\n%s\n' "$base_url" "$headers" >> "$SCAN_FILE"

    if [[ -z "$headers" ]]; then
        log_warn "No response from $base_url"
        return
    fi

    local server_hdr powered_hdr
    server_hdr="$(printf '%s' "$headers" | grep -i '^Server:' | tr -d '\r')"
    powered_hdr="$(printf '%s' "$headers" | grep -i '^X-Powered-By:' | tr -d '\r')"
    [[ -n "$server_hdr" ]] && log_ok "$server_hdr"
    [[ -n "$powered_hdr" ]] && log_ok "$powered_hdr"

    if [[ -n "$server_hdr" ]] || [[ -n "$powered_hdr" ]]; then
        log_finding "LOW" "Server/technology banner disclosed on port $port" \

```

```

        "curl -skI $base_url/ -> $(printf '%s %s' "$server_hdr" "$powered_hdr" | xargs)" \
        "Suppress or generalize the Server/X-Powered-By headers (ServerTokens/ServerSignature
in Apache, expose_php off in PHP, etc.)."
    fi

    local sec_headers_missing=()
    for h in "Content-Security-Policy" "X-Frame-Options" "X-Content-Type-Options" "Strict-
Transport-Security"; do
        printf '%s' "$headers" | grep -qi "^${h}:" || sec_headers_missing+=("$h")
    done
    if (( ${#sec_headers_missing[@]} > 0 )); then
        log_finding "LOW" "Missing security headers on port $port" \
            "curl -skI $base_url/ -> missing: ${sec_headers_missing[*]}" \
            "Add the missing security headers appropriate to the application (CSP, X-Frame-
Options, X-Content-Type-Options, HSTS for HTTPS)."
```

```

    fi

    # --- robots.txt ---
    local robots
    robots="$(timeout 8 curl -sk --max-time 6 "$base_url/robots.txt" 2>&1)"
    printf '\n=== robots.txt: %s/robots.txt ===\n%s\n' "$base_url" "$robots" >> "$SCAN_FILE"

    if [[ -n "$robots" ]] && ! printf '%s' "$robots" | grep -qi '404 Not Found\|<html'; then
        local disallow_count
        disallow_count="$(printf '%s' "$robots" | grep -ic '^Disallow:')"
        if (( disallow_count > 0 )); then
            local paths
            paths="$(printf '%s' "$robots" | grep -i '^Disallow:' | head -n 5 | tr '\n' ';' | sed
's/;/; /g')"
```

```

            log_finding "INFO" "robots.txt discloses $disallow_count restricted path(s) on port
$port" \
                "curl $base_url/robots.txt -> ${paths:0:200}" \
                "Not a vulnerability by itself, but treat disclosed paths as a roadmap for
further authorized testing; ensure they're properly access-controlled server-side."
        fi
    else
        log_ok "No robots.txt found (or empty)"
    fi

    # --- lightweight common-path probe (kept small & polite; not a full brute force) ---
    local common_paths=("admin" "login" "wp-admin" ".git/HEAD" "phpmyadmin" "config.php.bak")
    local found_paths=()
    local p code
    for p in "${common_paths[@]}; do
        code="$(timeout 6 curl -sk -o /dev/null -w '%{http_code}' --max-time 5 "$base_url/$p")"
        [[ "$code" =~ ^(200|301|302|403)$ ]] && found_paths+=("$p [$code]")
    done
    if (( ${#found_paths[@]} > 0 )); then
        printf '\n=== Common path probe: %s ===\n%s\n' "$base_url" "${found_paths[*]}" >>
"$SCAN_FILE"
        log_finding "INFO" "Potentially interesting paths found on port $port" \
            "curl probe of common paths on $base_url -> ${found_paths[*]}" \
            "Manually review each path's access control and whether it should be publicly
reachable."
    fi
}

=====
FILE: modules/portscan.sh
=====
#!/usr/bin/env bash
# =====
# portscan.sh – Stage 2: Port & Service Enumeration
# =====
# Populates two global associative arrays that the rest of the tool relies on:

```

```

# OPEN_PORTS[port]    = service name    (e.g. OPEN_PORTS[21]="ftp")
# PORT_VERSION[port]  = version/banner string reported by nmap -sV

portscan_menu() {
    local root_note=""
    (( EUID != 0 )) && root_note=" (needs sudo - will fall back to connect scan)"

    ask_menu "Port scan type - how should we scan $1?" 1 \
        "Fast      - top 100 ports, service/version detection (recommended default)" \
        "Full      - all 65535 TCP ports, service/version detection (slow)" \
        "Custom    - you specify a port range" \
        "Stealth SYN scan${root_note}" \
        "TCP connect scan only - no version detection (fastest, least info)"
    SCAN_TYPE="$MENU_CHOICE"

    if [[ "$SCAN_TYPE" == "3" ]]; then
        read -r -p "    Enter port range (e.g. 1-1000 or 22,80,443): " CUSTOM_PORT_RANGE
        [[ -z "$CUSTOM_PORT_RANGE" ]] && CUSTOM_PORT_RANGE="1-1000"
    fi
}

_build_nmap_args() {
    local args=(-Pn)
    case "$SCAN_TYPE" in
        1) args+=(-sV --top-ports 100) ;;
        2) args+=(-sV -p-) ;;
        3) args+=(-sV -p "$CUSTOM_PORT_RANGE") ;;
        4)
            if (( EUID == 0 )); then
                args+=(-sS -sV --top-ports 100)
            else
                log_warn "Stealth SYN scan requires root - falling back to TCP connect scan"
                args+=(-sT -sV --top-ports 100)
            fi
            ;;
        5) args+=(-sT --top-ports 100) ;;
        *) args+=(-sV --top-ports 100) ;;
    esac
    printf '%s\n' "${args[@]}"
}

run_portscan() {
    local target="$1"
    section "Stage 2/4 - Port & Service Enumeration: $target"

    local -a nmap_args
    mapfile -t nmap_args <<(_build_nmap_args)

    # --- ADDITIVE: apply firewall-evasion / protocol / stealth-profile choices
    #   from evasion.sh on top of the base args above (see modules/evasion.sh) ---
    apply_evasion_overrides nmap_args

    section "Scan Profile"
    log_info "Firewall evasion: $([[ "$FW_EVADE" == "1" ]] && echo "Yes (Xmas scan)" || echo
"No)"
    log_info "Protocol: ${SCAN_PROTOCOL^^}"
    log_info "Operational profile: $([[ "$STEALTH_PROFILE" == "redteam" ]] && echo "Red Team
(maximum stealth)" || echo "Penetration Test (standard)")"
    log_info "Technique: $SCAN_TECHNIQUE_LABEL"
    {
        printf '\n=== SCAN PROFILE: %s ===\n' "$target"
        scan_profile_banner_text
    } >> "$SCAN_FILE"

    log_info "Running: nmap ${nmap_args[*]} -oG - $target"
}

```

```

log_info "This can take a moment depending on scan type..."

local grep_out
grep_out="$(nmap "${nmap_args[@]}" -oG - "$target" 2>>"$LOG_FILE")"

{
    printf '\n=== PORT SCAN: %s ===\n' "$target"
    printf 'nmap %s -oG - %s\n\n' "${nmap_args[*]}" "$target"
    printf '%s\n' "$grep_out"
} >> "$SCAN_FILE"

local ports_line
ports_line="$(printf '%s\n' "$grep_out" | grep -F 'Ports:')"

if [[ -z "$ports_line" ]]; then
    log_warn "No open ports detected"
    return 0
fi

# "Ports: 21/open/tcp//ftp//vsftpd 2.3.4/, 22/open/tcp//ssh//OpenSSH 4.7p1/"
local ports_field entry port state proto owner service rpcinfo version
ports_field="${ports_line#*Ports: }"
IFS=',' read -ra entries <<< "$ports_field"

printf '\n%-8s %-8s %-10s %-40s\n' "PORT" "STATE" "SERVICE" "VERSION" >> "$SCAN_FILE"
printf '%s\n' "-----" >> "$SCAN_FILE"

for entry in "${entries[@]}"; do
    entry="$(echo "$entry" | xargs)" # trim whitespace
    # nmap -oG format: port/state/protocol/owner/service/rpc_info/version/
    IFS='/' read -r port state proto owner service rpcinfo version _ <<< "$entry"
    [[ "$state" != "open" ]] && continue
    [[ -z "$service" ]] && service="unknown"
    [[ -z "$version" ]] && version="(no banner captured)"

    OPEN_PORTS["$port"]="$service"
    PORT_VERSION["$port"]="$version"

    printf '%-8s %-8s %-10s %-40s\n' "$port" "$state" "$service" "$version" >> "$SCAN_FILE"
    log_ok "Port $port/$proto open - $service ${C_DIM}($version)${C_RESET}"
done

if (( ${#OPEN_PORTS[@]} == 0 )); then
    log_warn "Ports were reported but none parsed as open - check $SCAN_FILE for raw output"
else
    log_ok "${#OPEN_PORTS[@]} open port(s) recorded"
fi
}

=====
FILE: modules/recon.sh
=====
#!/usr/bin/env bash
# =====
# recon.sh - Stage 1: Reconnaissance
# =====

recon_menu() {
    ask_menu "Recon depth - how thorough should reconnaissance be?" 2 \
        "Quick - reachability + hostname resolution only" \
        "Standard - quick + traceroute + local ARP/route info (recommended)" \
        "Deep - standard + whois lookup" \
        "Skip recon entirely"
    RECON_LEVEL="$MENU_CHOICE"
}

```

```

}

run_recon() {
    local target="$1"
    section "Stage 1/4 – Reconnaissance: $target"

    if [[ "$RECON_LEVEL" == "4" ]]; then
        log_warn "Recon skipped by user choice"
        return 0
    fi

    {
        printf '=== RECON: %s ===\n' "$target"
        printf 'Date: %s\n\n' "$(date '+%Y-%m-%d %H:%M:%S')"
    } >> "$SCAN_FILE"

    # --- host availability -----
    log_info "Checking host availability (ping)..."
    local ping_out
    ping_out=$(ping -c 3 -W 2 "$target" 2>&1)
    echo "$ping_out" >> "$SCAN_FILE"

    if ! check_reachable "$target"; then
        log_fail "Target is unreachable"
        echo "$ping_out" | tail -n 3
        return "$EXIT_UNREACHABLE"
    fi
    log_ok "Target is reachable"

    # --- hostname / IP resolution -----
    log_info "Resolving IP / hostname information..."
    local ip host
    ip=$(resolve_target "$target")
    TARGET_IP="$ip"
    printf 'Resolved IP: %s\n' "${ip:-unknown}" | tee -a "$SCAN_FILE" >/dev/null
    log_ok "Resolved IP: ${ip:-unknown}"

    if command -v host &>/dev/null; then
        host=$(host "$ip" 2>/dev/null | head -n1)
        [[ -n "$host" ]] && { printf 'Reverse DNS: %s\n' "$host" >> "$SCAN_FILE"; log_ok "Reverse
DNS: $host"; }
    fi

    if [[ "$RECON_LEVEL" == "1" ]]; then
        return 0
    fi

    # --- basic network info (route / ARP) -----
    log_info "Gathering basic local network information..."
    {
        printf '\n-- Local route to target --\n'
        ip route get "$ip" 2>/dev/null
        printf '\n-- ARP table entry (if on local segment) --\n'
        arp -n 2>/dev/null | grep -F "$ip" || printf '(no ARP entry yet – will populate after
first packet exchange)\n'
    } >> "$SCAN_FILE"
    log_ok "Local network info captured"

    if command -v traceroute &>/dev/null; then
        log_info "Running traceroute (max 10 hops, 2s timeout)..."
        {
            printf '\n-- Traceroute --\n'
            traceroute -m 10 -w 2 "$ip" 2>&1
        } >> "$SCAN_FILE"
        log_ok "Traceroute captured"
    fi
}

```

```

else
    log_warn "traceroute not installed – skipping hop analysis"
fi

if [[ "$RECON_LEVEL" == "3" ]]; then
    if tool_available whois; then
        log_info "Running whois lookup..."
        { printf '\n-- Whois --\n'; whois "$ip" 2>&1 | head -n 40; } >> "$SCAN_FILE"
        log_ok "Whois captured"
    else
        log_warn "whois not installed – skipping (this is expected/normal for private-range
lab IPs)"
    fi
fi

return 0
}

```

```

=====
FILE: modules/report.sh
=====

```

```

#!/usr/bin/env bash

```

```

# =====
# report.sh – Stage 4: Report generation
# =====

```

```

report_menu() {
    ask_menu "Report output – what should we generate?" 2 \
        "Plain text only (scan.txt, findings.txt, summary.txt)" \
        "Plain text + HTML report (recommended)" \
        "Plain text + HTML + keep raw nmap greppable output"
    REPORT_MODE="$MENU_CHOICE"
}

```

```

_severity_rank() {
    case "$1" in
        HIGH) echo 3 ;; MEDIUM) echo 2 ;; LOW) echo 1 ;; *) echo 0 ;;
    esac
}

```

```

generate_findings_file() {
    {
        printf 'FINDINGS – %s\n' "$TARGET"
        printf 'Generated: %s\n' "$(date '+%Y-%m-%d %H:%M:%S')"
        printf '%s\n\n' "===== "

        if (( ${#FINDINGS[@]} == 0 )); then
            printf 'No findings were recorded for this target.\n'
        else
            local f severity title evidence recommendation n=1
            for f in "${FINDINGS[@]"; do
                IFS='|' read -r severity title evidence recommendation <<< "$f"
                printf '[%d] SEVERITY: %s\n' "$n" "$severity"
                printf '    Title: %s\n' "$title"
                printf '    Evidence: %s\n' "$evidence"
                printf '    Recommendation: %s\n\n' "$recommendation"
                ((n++))
            done
        fi
    } > "$FINDINGS_FILE"
}

```

```

generate_summary_file() {
    local high=0 medium=0 low=0 info=0
    local f severity

```

```

for f in "${FINDINGS[@]}"; do
    severity="${f%|*}"
    case "$severity" in
        HIGH) ((high++)) ;; MEDIUM) ((medium++)) ;; LOW) ((low++)) ;; INFO) ((info++)) ;;
    esac
done

local risk="Low"
(( medium > 0 )) && risk="Medium"
(( high > 0 )) && risk="High"

{
    printf 'SECURITY ASSESSMENT SUMMARY\n'
    printf '=====\n'
    printf 'Target:      %s\n' "$TARGET"
    printf 'Resolved IP:  %s\n' "${TARGET_IP:-n/a}"
    printf 'Scan date:    %s\n' "$(date '+%Y-%m-%d %H:%M:%S')"
    printf 'Duration:     %s\n' "$(human_duration "$SCAN_DURATION")"
    printf '\nScan profile:\n'
    printf '  Firewall evasion:      %s\n' "$([[ "$FW_EVADE" == "1" ]] && echo "Yes (Xmas
scan)" || echo "No")"
    printf '  Protocol:             %s\n' "${SCAN_PROTOCOL^^}"
    printf '  Operational profile:  %s\n' "$([[ "$STEALTH_PROFILE" == "redteam" ]] && echo
"Red Team (maximum stealth)" || echo "Penetration Test (standard)")"
    printf '  Technique used:       %s\n' "$SCAN_TECHNIQUE_LABEL"
    printf '  Stealth measures:     %s\n' "$STEALTH_LABEL"
    printf '\nOpen ports (%d):\n' "${#OPEN_PORTS[@]}"
    if (( ${#OPEN_PORTS[@]} > 0 )); then
        for p in "${!OPEN_PORTS[@]}"; do
            printf '  - %-6s %-10s %s\n' "$p" "${OPEN_PORTS[$p]}" "${PORT_VERSION[$p]:-}"
        done | sort -n
    else
        printf '  (none detected)\n'
    fi
    printf '\nFindings by severity:\n'
    printf '  HIGH:    %d\n  MEDIUM: %d\n  LOW:    %d\n  INFO:    %d\n' "$high" "$medium"
"$low" "$info"
    printf '\nOverall risk rating: %s\n' "$risk"
    printf '\nRisk explanation:\n'
    case "$risk" in
        High) printf '  At least one HIGH severity, evidence-backed finding was confirmed.
\n  These typically allow unauthorized access or information disclosure\n  and should be
remediated before this host is exposed further.\n' ;;
        Medium) printf '  No HIGH severity findings, but MEDIUM findings indicate
weaknesses\n  (e.g. outdated software, information disclosure) that reduce the\n  effort needed
for further compromise and should be scheduled for fixes.\n' ;;
        *) printf '  No HIGH/MEDIUM findings confirmed during this assessment. This
does\n  not guarantee the host is fully secure – it reflects only the checks\n  this tool
performed.\n' ;;
    esac
    printf '\nRecommendations:\n'
    if (( ${#FINDINGS[@]} == 0 )); then
        printf '  - Maintain current patching and configuration hygiene.\n  - Consider deeper
manual testing beyond this tool''''s automated checks.\n'
    else
        local f severity title _e recommendation
        for f in "${FINDINGS[@]}"; do
            IFS='|' read -r severity title _e recommendation <<< "$f"
            printf '  - [%s] %s: %s\n' "$severity" "$title" "$recommendation"
        done
    fi
    printf '\n-----\n'
    printf 'Generated by KarScan (karscan.sh) – Instant Software Solutions security track\n'
} > "$SUMMARY_FILE"
}

```

```

generate_html_report() {
    local high=0 medium=0 low=0 info=0
    local f severity
    for f in "${FINDINGS[@]"; do
        severity="${f%|*}"
        case "$severity" in
            HIGH) ((high++)) ;; MEDIUM) ((medium++)) ;; LOW) ((low++)) ;; INFO) ((info++)) ;;
        esac
    done
    local risk="Low" risk_color="#2e7d32"
    (( medium > 0 )) && { risk="Medium"; risk_color="#e6a700"; }
    (( high > 0 )) && { risk="High"; risk_color="#c0392b"; }

    {
cat <<HTML_HEAD
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Security Assessment Report – ${TARGET}</title>
<style>
    :root { color-scheme: light; }
    body { font-family: -apple-system, Segoe UI, Roboto, sans-serif; background:#f5f6f8;
color:#1b1f24; margin:0; padding:40px 20px; }
    .wrap { max-width: 900px; margin: 0 auto; }
    header { border-bottom: 3px solid #c0392b; padding-bottom: 16px; margin-bottom: 24px; }
    h1 { margin: 0 0 4px 0; font-size: 26px; }
    .meta { color:#5a636e; font-size: 14px; }
    .badge { display:inline-block; padding:4px 12px; border-radius:20px; color:#fff; font-
weight:600; font-size:13px; }
    .card { background:#fff; border:1px solid #e2e5e9; border-radius:10px; padding:20px 24px;
margin-bottom:20px; box-shadow:0 1px 2px rgba(0,0,0,.04); }
    table { width:100%; border-collapse: collapse; font-size: 14px; }
    th, td { text-align:left; padding:8px 10px; border-bottom:1px solid #eee; vertical-align: top;
}
    th { color:#5a636e; font-weight:600; }
    .sev { display:inline-block; padding:2px 10px; border-radius: 12px; color:#fff; font-size:12px;
font-weight:700; }
    .sev-HIGH { background:#c0392b; } .sev-MEDIUM { background:#e6a700; } .sev-LOW {
background:#3468c0; } .sev-INFO { background:#7a8390; }
    .evidence { font-family: ui-monospace, SFMono-Regular, Menlo, monospace; font-size: 12.5px;
background:#f2f3f5; padding:8px 10px; border-radius:6px; display:block; margin-top:4px; white-
space:pre-wrap; word-break: break-word; }
    footer { text-align:center; color:#9a9a99; font-size:12px; margin-top:30px; }
    .stat-row { display:flex; gap:16px; flex-wrap:wrap; }
    .stat { flex:1; min-width:110px; background:#fff; border:1px solid #e2e5e9; border-radius:10px;
padding:14px 18px; text-align:center; }
    .stat .num { font-size:26px; font-weight:700; }
</style>
</head>
<body>
<div class="wrap">
<header>
    <h1>Security Assessment Report</h1>
    <div class="meta">Target: <strong>${TARGET}</strong> (${TARGET_IP:-n/a}) &middot; Generated
    ${SCAN_DATE_HUMAN}</div>
    <span class="badge" style="background:${risk_color}">Overall risk: ${risk}</span>
</header>

<div class="stat-row">
    <div class="stat"><div class="num">${#OPEN_PORTS[@]}</div>Open ports</div>
    <div class="stat"><div class="num" style="color:#c0392b">${high}</div>High</div>
    <div class="stat"><div class="num" style="color:#e6a700">${medium}</div>Medium</div>
    <div class="stat"><div class="num" style="color:#3468c0">${low}</div>Low</div>

```

```

    <div class="stat"><div class="num" style="color:#7a8390">${info}</div>Info</div>
</div>

<div class="card">
    <h2>Scan Profile</h2>
    <table>
        <tr><th>Firewall evasion</th><td>${[[ "$FW_EVADE" == "1" ]] && echo "Yes (Xmas scan)" || echo
"No")</td></tr>
        <tr><th>Protocol</th><td>${SCAN_PROTOCOL^^}</td></tr>
        <tr><th>Operational profile</th><td>${[[ "$STEALTH_PROFILE" == "redteam" ]] && echo "Red Team
(maximum stealth)" || echo "Penetration Test (standard)"}</td></tr>
        <tr><th>Technique used</th><td>${_html_escape "$SCAN_TECHNIQUE_LABEL"}</td></tr>
        <tr><th>Stealth measures</th><td>${_html_escape "$STEALTH_LABEL"}</td></tr>
    </table>
</div>

<div class="card">
    <h2>Open Ports & Services</h2>
    <table>
        <tr><th>Port</th><th>Service</th><th>Version / Banner</th></tr>
HTML_HEAD

        if (( ${#OPEN_PORTS[@]} > 0 )); then
            for p in "${!OPEN_PORTS[@]"; do
                printf '<tr><td>%s</td><td>%s</td><td>%s</td></tr>\n' "$p" "${OPEN_PORTS[$p]}"
"${_html_escape "${PORT_VERSION[$p]:-}"})"
                done | sort -n
            else
                printf '<tr><td colspan="3">No open ports detected</td></tr>\n'
            fi
        fi

cat <<'HTML_MID'
    </table>
</div>

<div class="card">
    <h2>Findings</h2>
HTML_MID

    if (( ${#FINDINGS[@]} == 0 )); then
        printf '<p>No findings were recorded for this target.</p>\n'
    else
        local n=1 title evidence recommendation
        for f in "${FINDINGS[@]"; do
            IFS='|' read -r severity title evidence recommendation <<< "$f"
            cat <<FINDING_HTML
            <div style="padding:12px 0; border-top:1px solid #f0f0f0;">
                <span class="sev sev-${severity}">${severity}</span>
                <strong style="margin-left:8px;">${_html_escape "$title"}</strong>
                <span class="evidence">${_html_escape "$evidence"}</span>
                <div style="margin-top:6px; color:#4a525c; font-size:13.5px;"><strong>Recommendation:</
strong> ${_html_escape "$recommendation"}</div>
            </div>
            FINDING_HTML
            ((n++))
        done
    fi

cat <<'HTML_TAIL'
</div>
<footer>Generated by KarScan (karscan.sh) – Instant Software Solutions security track</footer>
</div>
</body>
</html>
HTML_TAIL

```

```

    } > "$HTML_FILE"
}

_html_escape() {
    local s="$1"
    s="${s//&/&#amp;}"; s="${s//</&lt;}"; s="${s//>/&gt;}"; s="${s//\"/\"&quot;}";
    printf '%s' "$s"
}

generate_reports() {
    section "Stage 4/4 – Report Generation"
    SCAN_DATE_HUMAN="$(date '+%Y-%m-%d %H:%M:%S')"

    generate_findings_file
    log_ok "Findings written to $FINDINGS_FILE"

    generate_summary_file
    log_ok "Summary written to $SUMMARY_FILE"

    log_ok "Raw scan log written to $SCAN_FILE"

    if [[ "$REPORT_MODE" != "1" ]]; then
        generate_html_report
        log_ok "HTML report written to $HTML_FILE"
    fi

    printf '\n'
    banner "Report ready: $REPORT_DIR"
}

=====
FILE: modules/smb.sh
=====
#!/usr/bin/env bash
# =====
# smb.sh – SMB service checks
# =====

check_smb() {
    local target="$1" port="$2"
    log_info "Starting SMB share enumeration on port $port..."

    local output=""

    if tool_available smbclient; then
        output="$(timeout 15 smbclient -N -L "//$target" -p "$port" 2>&1)"
        printf '\n=== smbclient -L //s ===\n%s\n' "$target" "$output" >> "$SCAN_FILE"
    elif tool_available enum4linux; then
        output="$(timeout 25 enum4linux -S "$target" 2>&1)"
        printf '\n=== enum4linux -S %s ===\n%s\n' "$target" "$output" >> "$SCAN_FILE"
    else
        log_warn "Neither smbclient nor enum4linux installed – falling back to nmap smb-enum-
shares script"
        output="$(timeout 30 nmap -p "$port" --script smb-enum-shares -Pn "$target" 2>&1)"
        printf '\n=== nmap smb-enum-shares on %s ===\n%s\n' "$target" "$output" >> "$SCAN_FILE"
    fi

    if [[ -z "$output" ]]; then
        log_warn "No SMB enumeration output obtained"
        return
    fi

    local shares
    shares="$(printf '%s' "$output" | grep -iE 'Disk|IPC|Sharename' | grep -viE '^Sharename' ||
true)"

```

```

if printf '%s' "$output" | grep -qiE 'NT_STATUS_ACCESS_DENIED' && [[ -z "$shares" ]]; then
    log_ok "SMB share listing requires authentication (access denied for null/guest session)"
    return
fi

if [[ -n "$shares" ]]; then
    local shares_line
    shares_line="$(printf '%s' "$shares" | tr '\n' ';' | sed 's/;/;/g' | cut -c1-200)"
    log_finding "MEDIUM" "SMB shares listable without authentication on port $port" \
        "smbclient -N -L //$target -p $port -> shares: $shares_line" \
        "Disable null/guest SMB sessions; require authentication for all shares and restrict
share permissions to least privilege."
    else
        log_ok "No shares listed via null/guest SMB session"
    fi
}

```

```

=====
FILE: modules/smtp.sh
=====

```

```

#!/usr/bin/env bash

```

```

# =====
# smtp.sh - SMTP service checks
# =====

```

```

check_smtp() {
    local target="$1" port="$2"
    log_info "Starting SMTP checks on port $port..."

    # --- banner ---
    local banner
    banner="$(timeout 5 bash -c "exec 3<>/dev/tcp/$target/$port; head -c 256 <&3 2>/dev/null)"
    printf '\n=== SMTP banner on %s:%s ===\n%s\n' "$target" "$port" "$banner" >> "$SCAN_FILE"
    [[ -n "$banner" ]] && log_ok "SMTP banner: ${banner//'\r'/}"

    # --- user enumeration via VRFY ---
    local vrfy_root vrfy_bogus
    vrfy_root="$(timeout 5 bash -c "
        exec 3<>/dev/tcp/$target/$port
        head -c 512 <&3 >/dev/null
        printf 'VRFY root\r\n' >&3
        timeout 3 head -c 256 <&3
    " 2>/dev/null)"

    vrfy_bogus="$(timeout 5 bash -c "
        exec 3<>/dev/tcp/$target/$port
        head -c 512 <&3 >/dev/null
        printf 'VRFY zzz_nonexistent_user_9182\r\n' >&3
        timeout 3 head -c 256 <&3
    " 2>/dev/null)"

    {
        printf '\n-- VRFY root --\n%s\n' "$vrfy_root"
        printf '\n-- VRFY bogus user --\n%s\n' "$vrfy_bogus"
    } >> "$SCAN_FILE"

    if printf '%s' "$vrfy_root" | grep -qE '^25[02]' && printf '%s' "$vrfy_bogus" | grep -qE
'^55[0-9]'; then
        log_finding "MEDIUM" "SMTP user enumeration via VRFY on port $port" \
            "VRFY root -> \"${vrfy_root//'\r\n'/}\" (accepted); VRFY zzz_nonexistent_user_9182
-> \"${vrfy_bogus//'\r\n'/}\" (rejected) - distinguishable responses confirm valid usernames."
        \
            "Disable the VRFY/EXPN commands (disable_vrfy_command in Postfix, or equivalent) so
responses do not confirm account existence."
    fi
}

```

```

else
    log_ok "SMTP VRFY does not appear to distinguish valid/invalid users"
fi

# --- open relay check (safe: RCPT TO two external domains, never DATA/send) ---
local relay_test
relay_test="$(timeout 6 bash -c "
    exec 3<>/dev/tcp/$target/$port
    head -c 512 <&3 >/dev/null
    printf 'HELO karscan.local\r\n' >&3; timeout 2 head -c 256 <&3 >/dev/null
    printf 'MAIL FROM:<probe@karscan.local>\r\n' >&3; timeout 2 head -c 256 <&3 >/dev/null
    printf 'RCPT TO:<relaytest@external-domain-example.test>\r\n' >&3
    timeout 2 head -c 256 <&3
    printf 'QUIT\r\n' >&3
" 2>/dev/null)"

    printf '\n-- Open relay probe (RCPT TO external domain) --\n%s\n' "$relay_test" >>
"$SCAN_FILE"

    if printf '%s' "$relay_test" | grep -qE '^25[02]'; then
        log_finding "HIGH" "Possible open mail relay on port $port" \
            "MAIL FROM:<probe@karscan.local> / RCPT TO:<relaytest@external-domain-example.test> -
> server replied 2xx (accepted) for an external, unauthenticated recipient." \
            "Restrict relaying to authenticated users / trusted networks only
(smtpd_relay_restrictions); re-test with a real external mailbox before treating this as
confirmed."
    else
        log_ok "Server rejected unauthenticated relay attempt to an external domain"
    fi
}

=====
FILE: modules/ssh.sh
=====
#!/usr/bin/env bash
# =====
# ssh.sh - SSH service checks
# =====

# A small, well-known set of end-of-life / historically vulnerable OpenSSH
# banners used only to flag "this looks outdated, verify the CVE yourself" -
# never treated as a confirmed exploit.
_ssh_is_old_banner() {
    # Accepts both the raw banner form (OpenSSH_5.3) and the nmap-formatted
    # fallback form (OpenSSH 5.3p1) - the space/underscore differs depending
    # on whether we grabbed the banner live or fell back to nmap's -sV data.
    local banner="$1"
    [[ "$banner" =~ OpenSSH[_ ] [1-6]\. ]] && return 0
    [[ "$banner" =~ OpenSSH[_ ] 7\.[0-4] ]] && return 0
    return 1
}

check_ssh() {
    local target="$1" port="$2"
    log_info "Starting SSH banner check on port $port..."

    local banner
    banner="$(timeout 5 bash -c "exec 3<>/dev/tcp/$target/$port; head -c 256 <&3" 2>/dev/null)"

    if [[ -z "$banner" ]]; then
        banner="${PORT_VERSION[$port]:-}"
    fi

    printf '\n=== SSH banner on %s:%s ===\n%s\n' "$target" "$port" "$banner" >> "$SCAN_FILE"

```

```
if [[ -z "$banner" ]]; then
    log_warn "Could not retrieve SSH banner"
    return
fi

log_ok "SSH banner: $banner"

if _ssh_is_old_banner "$banner"; then
    log_finding "MEDIUM" "Outdated OpenSSH version banner on port $port" \
        "Banner grabbed via raw TCP connect: \"${banner//$'\r'}/}\"" \
        "Upgrade OpenSSH to a currently supported release and check the banner's version
against the NVD for known CVEs."
    fi
}
```